

Fundamentos de la programación II

Laboratorio. Hoja 6

Los ejercicios marcados como **Evaluable** tienen que subirse al CV durante la sesión de laboratorio.

- Se subirá **únicamente el archivo Program.cs**
- Sólo hará la entrega uno de los miembros del grupo.
- La línea 1 (y siguientes) deben contener los nombres de los alumnos que hacen la entrega de la forma:

```
// Nombre Apellido1 Apellido2  
// Nombre Apellido1 Apellido2
```

1. Consideremos la clase de las listas enlazadas de enteros vista que se está desarrollando en clase (disponible en el CV). En este ejercicio se trata de enriquecerla con los nuevos métodos. Para cada uno de ellos deberá incluirse la opción correspondiente de menú en el programa principal, para probar la funcionalidad.

En muchos métodos es necesario utilizar variables locales de tipo `Nodo` para recorrer la lista apuntando a cada nodo, sin modificar la propia lista. Por ejemplo, el método `BuscaDato` se implementa de la siguiente forma:

```
public bool BuscaDato(int e){  
    Nodo aux = pri; // referencia auxiliar para recorrer la lista  
    // bucle de búsqueda en la lista  
    while (aux!=null && aux.dato!=e)  
        aux = aux.sig;  
    return aux!=null;  
}
```

Extender la clase de las listas enlazadas con los siguientes métodos:

- (ya implementado) `void InsertaPri(int e)`: inserta el elemento `e` al principio de la lista.
- (ya implementado) `bool BuscaDato(int e)`: devuelve `true` si el elemento `e` está en la lista y `false` en caso contrario.
- (**Evaluable**) `public int NumElem()`: devuelve el número de elementos de la lista.
- (**Evaluable**) `public int Suma()`: devuelve la suma de los elementos de la lista.
- `public int CuentaOcs(int e)`: devuelve el número de ocurrencias del elemento `e` en la lista.
- `bool InsertaPos(int e)`: inserta el elemento `e` en la posición `pos` de la lista. Si la posición no existe, no se inserta el elemento y se devuelve `false`; en caso contrario, se inserta el elemento y se devuelve `true`.
- `void InsertaUlt(int e)`: inserta el elemento `e` al final de la lista. Este método Debe implementarse con un solo recorrido de la lista.
- `bool BorraElto(int e)`: borra la primera aparición del elemento `e` en la lista. Devuelve `true` si se ha borrado el elemento y `false` en caso contrario.
- `private Nodo NesimoNodo(int n)`: devuelve el `n`-ésimo nodo de la lista; `null` si no existe tal elemento.
- `public int Nesimo(int n)`: utilizando el método anterior, devuelve el `n`-ésimo elemento de la lista; lanza una excepción en si no existe tal elemento.
- `public void InsertaNesimo(int n, int e)`: inserta el elemento `e` en el lugar `n`-ésimo de la lista (puede ser útil el método `Nesimo`).
- `public void BorraTodos(int e)`: borra todas las apariciones del elemento `e` en la lista.
- `public void BorraNesimo(int n)`: utilizando `NesimoNodo` borra el `n`-ésimo elemento de la lista, si existe.

- `public void Invierte():` invierte el orden de los elementos de la lista.
- `public bool Iguales(Lista l):` comprueba si la lista del objeto actual (`this`) es igual a `l`, i.e., tiene los mismos elementos en el mismo orden.
- `public bool Iguales2(Lista l):` comprueba si la lista del objeto actual (`this`) tiene los mismos elementos que `l` aunque sea en distinto orden.